

Universidade Federal Rural de Pernambuco

Resolva individualmente e entregue um PDF, digitado ou digitalizado a partir de manuscrito legível. Árvores podem ser desenhadas à mão, desde que os nós e as arestas estejam claros.

As gramáticas

As duas geram a mesma linguagem, com soma, divisão, parênteses, identificadores e números.

G_1 , usada na análise descendente

Sem recursão à esquerda e fatorada.

- (1) $E \rightarrow T E'$
- (2) $E' \rightarrow + T E'$
- (3) $E' \rightarrow \varepsilon$
- (4) $T \rightarrow F T'$
- (5) $T' \rightarrow / F T'$
- (6) $T' \rightarrow \varepsilon$
- (7) $F \rightarrow (E)$
- (8) $F \rightarrow \text{id}$
- (9) $F \rightarrow \text{num}$

G_2 , usada na análise ascendente

Recursiva à esquerda, como se escreve no Bison.

- (0) $E^* \rightarrow E$
- (1) $E \rightarrow E + T$
- (2) $E \rightarrow T$
- (3) $T \rightarrow T / F$
- (4) $T \rightarrow F$
- (5) $F \rightarrow (E)$
- (6) $F \rightarrow \text{id}$
- (7) $F \rightarrow \text{num}$

Os símbolos terminais são os tokens que o analisador léxico entrega: `id`, `num`, `+`, `/`, `(`, `)`. O marcador `$` indica o fim da entrada. Em `(a + b) / 2`, o léxico produz `( id + id ) / num`, é essa sequência que chega ao parser.

Tabela M de G_1

M	<code>id</code>	<code>num</code>	<code>+</code>	<code>/</code>	<code>(</code>	<code>)</code>	<code>\$</code>
E	$E \rightarrow T E'$	$E \rightarrow T E'$			$E \rightarrow T E'$		
E'			$E' \rightarrow + T E'$			$E' \rightarrow \varepsilon$	$E' \rightarrow \varepsilon$
T	$T \rightarrow F T'$	$T \rightarrow F T'$			$T \rightarrow F T'$		
T'			$T' \rightarrow \varepsilon$	$T' \rightarrow / F T'$		$T' \rightarrow \varepsilon$	$T' \rightarrow \varepsilon$
F	$F \rightarrow \text{id}$	$F \rightarrow \text{num}$			$F \rightarrow (E)$		

Célula em branco significa erro sintático.

Tabela ACTION/GOTO de G_2

Estado	ACTION							GOTO		
	id	num	+	/	(	)	\$	<i>E</i>	<i>T</i>	<i>F</i>
0	s5	s6			s4			1	2	3
1			s7				acc			
2			r2	s8		r2	r2			
3			r4	r4		r4	r4			
4	s5	s6			s4			9	2	3
5			r6	r6		r6	r6			
6			r7	r7		r7	r7			
7	s5	s6			s4				10	3
8	s5	s6			s4					11
9			s7			s12				
10			r1	s8		r1	r1			
11			r3	r3		r3	r3			
12			r5	r5		r5	r5			

Questão 1

Parser preditivo. Use a tabela *M* para analisar a entrada

id + num / id \$

Escreva o traço completo, uma linha por passo, até aceitar. As três primeiras linhas estão feitas:

#	Pilha	Entrada	Ação
1	\$ E	id + num / id \$	$E \rightarrow T E'$
2	\$ E' T	id + num / id \$	$T \rightarrow F T'$
3	\$ E' T' F	id + num / id \$	$F \rightarrow \text{id}$
4	...		

(a) Complete a tabela acima.

(b) Faça uma tabela para a entrada (num), mostrando o traço do parser preditivo. Quantos passos ele deu? Quantos tokens consumiu?

Questão 2

Parser SLR(1), mesma entrada. Agora use a tabela ACTION/GOTO sobre

id + num / id \$

São duas pilhas lado a lado. A de símbolos tem sempre um elemento a menos que a de estados, porque o estado 0 não tem símbolo embaixo dele.

#	Pilha de estados	Pilha de símbolos	Entrada	Ação
1	0	<i>vazia</i>	id + num / id \$	s5
2	0 5	id	+ num / id \$	r6: $F \rightarrow id$
3	0 3	F	+ num / id \$	r4: $T \rightarrow F$
4	...			

- (a) Complete o traço até **acc**.
- (b) No passo #4, o topo da pilha de estados é o 2 e o token adiante é um +. A tabela manda reduzir por $E \rightarrow T$. Repare que a mesma linha 2 manda empilhar quando o token é /. Explique o que muda na entrada para justificar as duas decisões diferentes no mesmo estado.
- (c) Liste as reduções na ordem em que aconteceram. Invertendo essa lista, que derivação você obtém?
- (d) Faça uma tabela para a entrada (**num**), mostrando o traço do parser SLR(1). Quantos passos ele deu? Quantas reduções? Compare com o número de passos que o parser preditivo gastou na mesma entrada, na questão 1.

Questão 3

(id + id) / num \$

Analise essa entrada com a tabela ACTION/GOTO.

- (a) Liste, na ordem, as reduções aplicadas. São onze.
- (b) Desenhe a árvore de derivação. Construa de baixo para cima, na ordem das reduções da letra (a).

Questão 4

Um aluno digitou **a + / 2**, que chega ao parser como

id + / num \$

- (a) Rode o parser preditivo até parar. Diga qual célula de M estava vazia e quantos tokens tinham sido consumidos até ali.
- (b) Rode o parser SLR(1) até parar. Diga qual célula de ACTION estava vazia e em que estado.
- (c) Os dois pararam no mesmo token da entrada ou em tokens diferentes?
- (d) Cada parser sabe dizer o que esperava encontrar. No caso do preditivo, essa informação vem do símbolo que estava no topo da pilha; no caso do SLR(1), vem da linha do estado

corrente. Escreva os dois conjuntos de tokens esperados e compare.

- (e) Escreva a mensagem de erro que você gostaria de ver, com o token encontrado e os esperados, no formato de uma linha só.